

Національний університет «Одеська Політехніка»

Інститут комп'ютерних систем

Кафедра інформаційних систем

Лабораторна робота №5

з дисципліни «Забезпечення якості та контроль якості і тестування ПП»

Тема: «Модульні (Unit) тести та Інтеграційні (Integration) тести.»

Виконав:

студент групи АІ-243

Гаврилов О.В.

Перевірили:

Баськов І.О.

Одеса 2026

Мета: Ознайомитися з основами модульного та інтеграційного тестування. Навчитися писати тести однією з обраних мов програмування (JavaScript) для окремих функцій та інтеграції між модулями. Освоїти використання бібліотек для тестування, таких як Jest, та підключення Test Management System (testomat.io).

Завдання:

- 1) Повторити покрокові інструкції зі створення проєкту, написання модульних та інтеграційних тестів за допомогою Jest.
- 2) Зафіксувати роботу у звіті (скріншоти структури проєкту, лістинг тестів та результати тестових прогонів у консолі).
- 3) Обрати власний проєкт для написання тестів.
- 4) Написати 3 модульних тести та 2 інтеграційних тести для обраного власного проєкту.
- 5) Зафіксувати роботу у звіті, оновити репозиторій на GitHub та надати посилання, підключити результати до системи testomat.io.

Хід виконання роботи:

- 1) Під час виконання лабораторної роботи у середовищі Visual Studio Code було ініціалізовано новий проєкт на базі Node.js за допомогою пакетного менеджера npm, а також встановлено тестовий фреймворк Jest та бібліотеку Axios для виконання HTTP-запитів;
- 2) На першому етапі було реалізовано модульне тестування: у директорії src створено базові функції для математичних обчислень (math.js) та валідації паролів (auth.js), а в директорії tests — відповідні скрипти перевірки, які успішно пройшли тестування;
- 3) На другому етапі проведено інтеграційне тестування: розроблено скрипти для перевірки спільної взаємодії функцій конвертації валют (currency.js), а також реалізовано тестування запитів до зовнішнього API (api.js) з використанням mock-об'єктів (імітацій) для ізоляції перевірок від реальної мережі;

4) Налаштовано інтеграцію тестового середовища з хмарною системою управління тестуванням Testomat.io за допомогою пакета `@testomat.io/reporter`, що дозволило успішно експортувати результати автоматизованих прогонів (Test Runs) до дашборду системи;

5) На фінальному етапі було обрано власний проєкт, для якого розроблено унікальні 3 модульні та 2 інтеграційні тести, після чого сирцевий код та тестові скрипти було завантажено до репозиторію на GitHub.

Завдання 1

Опис завдання: Створити проєкт на базі Node.js, встановити фреймворк Jest та бібліотеку Axios. Написати базові модульні та інтеграційні тести згідно з методичними вказівками та виконати їх у консолі.

Пояснення до виконання (Що було зроблено): У середовищі Visual Studio Code було ініціалізовано новий проєкт та налаштовано тестове середовище. У директорії src створено 4 базові модулі: math.js (додавання), auth.js (валідація пароля), currency.js (конвертація валют) та api.js (HTTP-запити). У директорії tests створено відповідні перевіірочні скрипти. Для ізольованого тестування HTTP-запитів використано mock-об'єкт `jest.mock('axios')`. Усі тести успішно пройшли перевірку, що підтверджується результатами в терміналі.

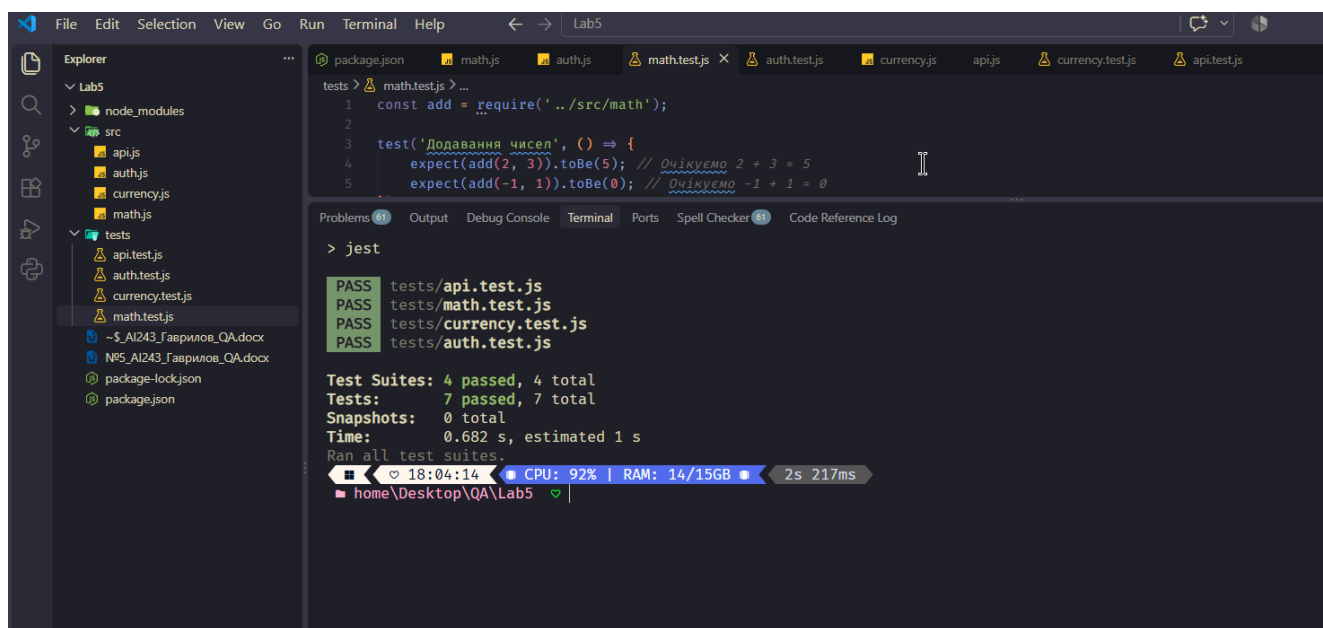


Рисунок 1 – Структура проєкту та результати успішного локального виконання тестів у терміналі

Завдання 2.

Опис завдання: Інтегрувати виконання тестів у систему управління тестуванням Testomat.io та експортувати результати.

Пояснення до виконання (Що було зроблено): Для накопичення даних про тестування було створено проєкт у системі Testomat.io. У локальний проєкт встановлено пакет `@testomat.io/reporter` та налаштовано файл `jest.config.js`. За допомогою згенерованої команди з API-ключем тестові скрипти було успішно імпортовано та виконано локально з автоматичною передачею результатів (Test Run) до хмарного дашборду системи.

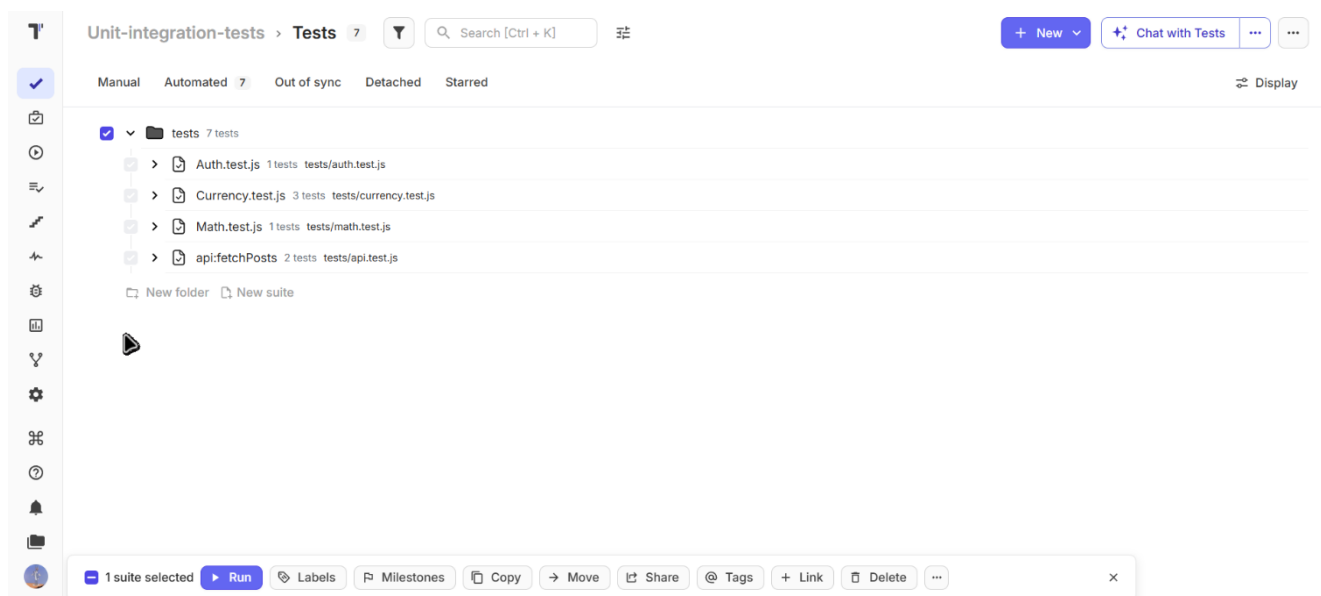


Рисунок 2 – Імпорт тестових сценаріїв до системи управління тестуванням Testomat.io

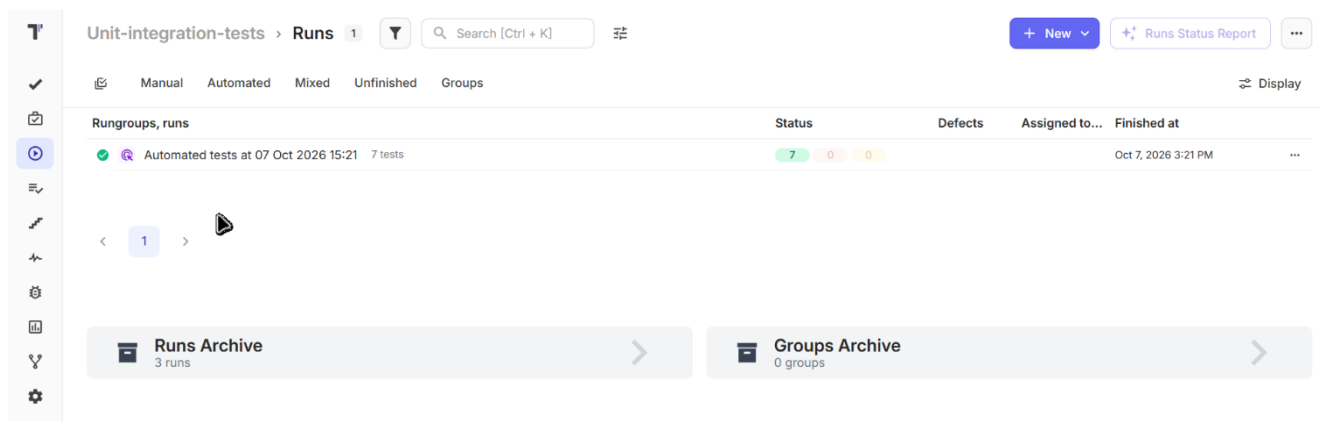


Рисунок 3 – Звіт про автоматизований запуск тестів (Test Run) у середовищі Testomat.io

Завдання 3, 4 та 5.

Опис завдання: Обрати або створити особистий проєкт. Написати для нього 3 модульних та 2 інтеграційних тести. Зафіксувати результати у звіті, додати лістинги коду та посилання на репозиторій у GitHub.

Пояснення до виконання (Що було зроблено): Для самостійного тестування обрано міні-проєкт «Система електронної комерції» (кошик товарів). Було написано 3 модульні тести, які перевіряють логіку математичного підрахунку суми, коректність віднімання знижки та правильність регулярного виразу для валідації email. Також написано 2 інтеграційні тести: перший перевіряє спільну роботу функцій підрахунку суми та знижки в процесі оформлення замовлення, а другий — використовує mock-об'єкт для імітації HTTP-запиту даних профілю користувача. Сирцевий код успішно протестовано та завантажено до репозиторію на GitHub.

Лістинг 1. Сирцевий код власного проєкту (файл src/myProject.js)

```
const axios = require('axios');

function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}
function applyDiscount(total, discountPercent) {
  return total - (total * (discountPercent / 100));
}
function validateEmail(email) {
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return re.test(email);
}
function checkout(items, discountPercent) {
  const total = calculateTotal(items);
  return applyDiscount(total, discountPercent);
}
async function getUserProfile(userId) {
  const response = await axios.get(`https://api.example.com/users/${userId}`);
  return response.data;
}
module.exports = { calculateTotal, applyDiscount, validateEmail, checkout,
  getUserProfile };
```

Лістинг 2. Кастомні тести власного проєкту (файл tests/myProject.test.js)

```
const { calculateTotal, applyDiscount, validateEmail, checkout, getUserProfile } =
  require('../src/myProject');
const axios = require('axios');
jest.mock('axios');
```

```

describe('Модульні тести', () => {
  test('calculateTotal: підрахунок суми', () => {
    expect(calculateTotal([ { price: 100 }, { price: 50 } ])).toBe(150);
  });
  test('applyDiscount: застосування знижки', () => {
    expect(applyDiscount(200, 10)).toBe(180);
  });
  test('validateEmail: перевірка email', () => {
    expect(validateEmail('test@mail.com')).toBe(true);
  });
});

describe('Інтеграційні тести', () => {
  test('checkout: сума та знижка разом', () => {
    expect(checkout([ { price: 100 }, { price: 100 } ], 20)).toBe(160);
  });
  test('getUserProfile: отримання профілю (Mock API)', async () => {
    axios.get.mockResolvedValue({ data: { id: 1, name: 'Олександр' } });
    const profile = await getUserProfile(1);
    expect(profile.name).toEqual('Олександр');
  });
});

```

```

Problems Output Debug Console Terminal Ports Spell Checker Code Reference Log
SHOWING JEST TESTS FROM **/*{.,_}{test,spec,cy}.js: ...
18:21:04 CPU: 90% | RAM: 13/15GB 5s 194ms 26.7.0
home\Desktop\QA\Lab5 npm test

> lab5@1.0.0 test
> jest

Determining test suites to run...[TESTOMATIO] Testomatio Reporter v2.18.0
[TESTOMATIO] Pipes: No pipes enabled
[TESTOMATIO] If you want to use Testomatio reporter, pass your token as TESTOMATIO env variable
PASS tests/myProject.test.js
  Модульні тести
    ✓ calculateTotal: підрахунок суми (2 ms)
    ✓ applyDiscount: застосування знижки
    ✓ validateEmail: перевірка email
  Інтеграційні тести
    ✓ checkout: сума та знижка разом
    ✓ getUserProfile: отримання профілю (Mock API) (1 ms)

PASS tests/api.test.js
  api:fetchPosts
    ✓ повертає пости з відповіді API (1 ms)
    ✓ передає помилку API викликувачу (3 ms)

PASS tests/currency.test.js
  ✓ Перетворення валюти у число
  ✓ Форматування числа у валюту
  ✓ Складання двох значень валюти (1 ms)

PASS tests/auth.test.js
  ✓ Перевірка валідності пароля (1 ms)

PASS tests/math.test.js
  ✓ Додавання чисел (1 ms)

Test Suites: 5 passed, 5 total
Tests: 12 passed, 12 total

```

Рисунок 4 – Результати виконання кастомних модульних та інтеграційних тестів власного проекту

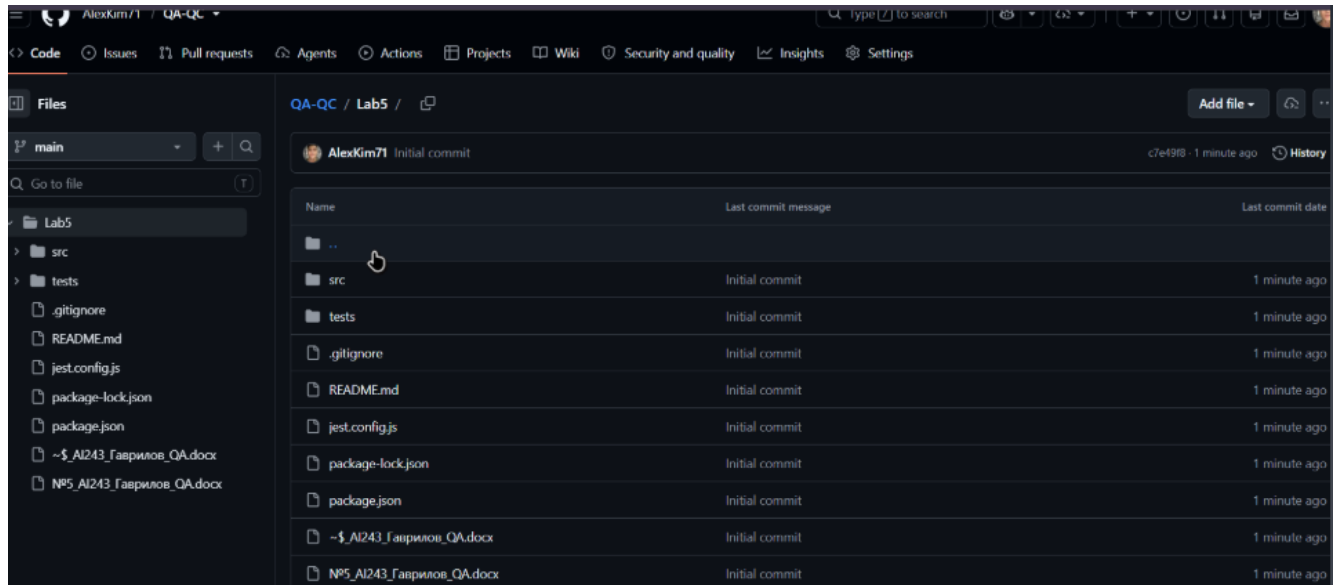


Рисунок 5 – Репозиторій із сирцевим кодом та тестами на платформі GitHub

Контрольні питання:

1) Що таке модульне тестування, і які його переваги? Модульне тестування (Unit testing) — це процес тестування окремих, найменших частин коду (функцій, методів, класів) в ізоляції від решти системи. Його головні переваги: швидке виявлення та локалізація помилок на ранніх етапах розробки, спрощення та безпека рефакторингу коду, а також те, що тести слугують додатковою технічною документацією до коду.

2) У чому полягає різниця між модульними та інтеграційними тестами? Модульні тести перевіряють роботу однієї ізольованої функції, ігноруючи її зв'язки з іншими компонентами (наприклад, чи правильно функція `add(2, 3)` повертає 5). Інтеграційні тести перевіряють взаємодію двох або більше модулів разом, а також роботу з базами даних, файловою системою чи зовнішніми API, щоб переконатися, що система працює як єдине ціле.

3) Як створити mock-об'єкт для тестування інтеграції? У середовищі Jest mock-об'єкт створюється за допомогою функції `jest.mock()`. Наприклад, для імітації запитів бібліотеки `Axios` використовується `jest.mock('axios')`, після чого за допомогою `axios.get.mockReturnValue()` можна підставити (замокати) штучні дані, які функція отримає замість реальної відповіді від сервера. Це дозволяє тестувати логіку обробки даних без залежності від інтернет-з'єднання.

4) Які бібліотеки використовуються для тестування у JavaScript? Найбільш популярними бібліотеками для написання та запуску тестів у JavaScript є Jest (надає все необхідне з коробки), Mocha (часто використовується в комбінації з бібліотекою тверджень Chai), а також Jasmine та Cypress (для наскрізного тестування).

5) Як перевірити коректність HTTP-запиту? Коректність HTTP-запиту перевіряється шляхом аналізу отриманої відповіді. В інтеграційних тестах перевіряють статус-код відповіді (наприклад, чи дорівнює він 200 OK), структуру отриманого JSON-об'єкта (чи містить він потрібні поля `toHaveProperty`), тип даних

(наприклад, чи є відповідь масивом), а також кількість викликів самої функції запиту через mock-об'єкти (наприклад, `toBeCalledTimes(1)`).

Висновок:

Під час виконання лабораторної роботи було успішно опановано основи модульного (Unit) та інтеграційного (Integration) тестування програмного забезпечення. За допомогою середовища Node.js та фреймворку Jest було створено та налаштовано тестове середовище, а також практично закріплено навички написання модульних тестів для перевірки ізольованих функцій та інтеграційних тестів для перевірки взаємодії компонентів. Окрему увагу було приділено використанню mock-об'єктів для імітації HTTP-запитів зовнішніх API (за допомогою бібліотеки Axios).

Важливим етапом стало налаштування інтеграції локального тестового фреймворку із хмарною системою управління тестуванням Testomat.io. Це дозволило автоматизувати експорт результатів тестових прогонів (Test Runs) та їхній моніторинг через аналітичний дашборд.

У межах самостійного завдання було розроблено власний міні-проект, для якого успішно написано та безпомилково виконано 3 модульні та 2 інтеграційні тести. Сирцевий код проекту та тестові сценарії було завантажено до репозиторію на GitHub для забезпечення надійного версіонування коду. Усі завдання виконані в повному обсязі, мету роботи досягнуто.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Власний репозиторій проєкту на платформі GitHub. URL: <https://github.com/AlexKim71/QA-QC> (дата звернення: 07.10.2026).
2. Проєкт автоматизованого тестування у системі Testomat.io. URL: <https://app.testomat.io/projects/unit-integration-tests-b6fbd/> (дата звернення: 07.10.2026).
3. Офіційна документація фреймворку Jest. URL: <https://jestjs.io/uk/docs/getting-started> (дата звернення: 07.10.2026).
4. Відеоматеріал: Інтеграція testomat.io з автоматизованими тестами. URL: <https://www.youtube.com/watch?v=HKbNn7gusI0> (дата звернення: 07.10.2026).
Офіційна документація тестового фреймворку Mocha. URL: <https://mochajs.org/> (дата звернення: 07.10.2026).